SARDAR PATEL INSTITUTE OF TECHNOLOGY

Munshi Nagar, Andheri (West), Mumbai

Department of Computer Engineering

LLM Production and Deployment

Project Report

Grounded Crop Disease Diagnosis and Treatment Advisory Using Retrieval-Augmented Language Models

Submitted by

Shivam Santosh Kadam

UID: 2023300099

Computer Engineering (COMPS) · Division B

Live application

https://crop-disease-rag-advisor.streamlit.app

Source code

https://github.com/ShivamKadam63s/crop-disease-rag-advisor

Deployed generator: ***openai****/gpt-oss-20b via* ***Groq*** *(https://api.groq.com/openai/v1)*

Academic Year 2026–27 · November 2026

Table of Contents

Abstract **ii**

1 Introduction **1**

1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 1
1.4 Contributions . . . 2
1.5 Technical Background . . . 2
1.6 Organisation of the Report . . . 3

2 Application Workflow **3**

2.1 System Architecture . . . 3
2.2 Knowledge Base Construction . . . 4
2.3 Hybrid Retrieval . . . 5
2.4 Prompting and Generation . . . 6
2.5 LangGraph Decision Layer . . . 6
2.6 Multilingual Input, Photo Input and Answer Control . . . 8
2.7 Model Variants, Fine-Tuning and Compression . . . 8
2.8 User Interface, API and Monitoring . . . 8

3 Results and Observations **10**

3.1 Experimental Setup . . . 10
3.2 Dataset and Retrieval Results . . . 11
3.3 Retrieval Gate Calibration and Service Load . . . 13
3.4 Fine-Tuning Results . . . 14
3.5 Generation Quality . . . 15
3.6 Safety: Out-of-Distribution Questions and Doses . . . 17
3.7 Efficiency and Compression Trade-offs . . . 18
3.8 Observations from the Live Application . . . 20
3.9 Summary of Findings . . . 23

4 Conclusion **24**

4.1 Conclusion . . . 24
4.2 Limitations . . . 24
4.3 Future Work . . . 24

References **25**

Abstract

Smallholder farmers often need to make decisions about treating crop diseases without having immediate access to an agronomist. Although general-purpose chatbots can provide quick responses, they may generate inaccurate information, such as incorrect disease names, unsuitable products or unsafe pesticide dosages. To address this problem, this project develops and evaluates a **grounded crop-disease diagnosis and treatment advisory system**. Farmers can describe crop symptoms using text or photographs in English, Hinglish, Hindi or Marathi. The system provides a probable diagnosis and treatment recommendations supported by citations from a curated knowledge base. If sufficient supporting information is unavailable, it provides a clear and helpful response instead of generating unsupported advice.

The retrieval and generation components are implemented as independent services. The retrieval system combines BGE-small embeddings with a FAISS index and BM25 using Reciprocal Rank Fusion. It also applies crop-based filtering and parent-chunk expansion to improve the relevance of retrieved information. A **LangGraph-based decision layer** manages the generation process through a scope guard, a calibrated retrieval threshold and a review stage. This review checks citation accuracy, evidence support and pesticide dosage values. If an answer fails these checks, the system allows one strict retry before selecting from six predefined fallback responses. The study evaluates six variants of Qwen2.5-0.5B-Instruct (FP16, INT8, NF4, LoRA, QLoRA and LoRA-then-NF4), both with and without retrieval. Evaluation is performed using 150 test questions, 36 handwritten questions, 40 terminology questions, 90 long-prompt questions and 40 out-of-distribution questions. The study also measures computational efficiency, perplexity and retrieval load.

The results show that retrieval-augmented prompting improves the test accuracy of all untuned model variants, increasing it from 0.19–0.23 to 0.54–0.66. **QLoRA with retrieval achieves the best performance among the locally tested variants**, with a test accuracy of 0.92, rare-class F1 score of 0.917 and citation correctness of 0.75. It produces no unsupported pesticide dosage recommendations and correctly refuses 95% of out-of-distribution questions. The model operates within 0.66G̃B of GPU memory and achieves consistent accuracy between 0.907 and 0.920 across three independent training runs. The decision layer introduces a trade-off between accuracy and safety, reducing test accuracy by 16 percentage points while achieving a false-accept rate of 4%. Finally, the project includes a Streamlit-based application that provides multiple answer styles, language selection, source highlighting, a workflow viewer and a monitoring dashboard.

Keywords: *retrieval-augmented generation, crop disease advisory, hybrid retrieval, LangGraph, LoRA, QLoRA, quantization, grounded generation, safety fallback.*

1. Introduction

1.1 Background and Motivation

Agriculture employs a large share of the Indian workforce, and crop losses to disease and pests directly affect farm income and food supply. A farmer who notices spindle-shaped spots on rice seedlings, yellow stripes on wheat leaves or small holes in cotton bolls must decide quickly what the problem is and what to do. Trained advisers such as Krishi Vigyan Kendra staff and state agriculture officers exist, but they are few relative to the number of farms, and the Kisan Call Centre cannot always be reached at the moment a decision is needed. Large language models (LLMs) appear to offer an always-available adviser that can understand colloquial descriptions, including Hindi written in Roman letters.

The difficulty is that a language model answers from its parameters, not from a reviewed source. In agronomy this has three concrete consequences. First, the model can name a disease that does not match the symptoms, or a disease that does not occur on the crop. Second, it can invent a pesticide name or, more dangerously, a **spray dose**, and a wrong dose can damage the crop, harm the farmer or leave residues. Third, it rarely admits that a question is outside its competence: asked about a banana disease or a sick buffalo, it usually produces a confident answer. Retrieval-augmented generation (RAG) [1] addresses part of this by placing retrieved passages in the prompt so that answers can be cited, but retrieval alone does not stop a model from ignoring the passages, miscopying a number or answering an unrelated question.

Two practical constraints make the problem harder. Models small enough to run on a free GPU or a laptop are attractive for low-cost deployment, but they are weaker, and compressing them with quantization or adapting them with low-rank fine-tuning changes their behaviour in ways that average accuracy hides. The project is therefore motivated by a question that is as much about engineering as about modelling: **what combination of retrieval, fine-tuning, compression and runtime checks produces an advisor that is accurate, grounded, safe and cheap enough to deploy?**

1.2 Problem Statement

Design, implement, deploy and evaluate a crop-disease advisory system that accepts a farmer's free-text (or photo-derived) description of symptoms and returns (i) the most likely disease or pest from a closed list, (ii) treatment and management advice that is traceable to specific passages of a knowledge base, and (iii) an explicit, typed fallback whenever the question is out of scope, the crop is unsupported, the evidence is weak or a dose would have to be invented. The system must be evaluated not only on average accuracy but on rare classes, long and colloquial inputs, out-of-distribution questions and dose safety, and the compression and fine-tuning choices must be justified by measured quality, memory, latency and cost.

1.3 Objectives

- Build a versioned, validated knowledge base of structured disease cards, with section-level chunks whose character offsets make citations clickable and highlightable.
- Implement a hybrid retrieval service (dense + BM25 with Reciprocal Rank Fusion, crop filtering, alias expansion and parent expansion) that is independent of the generation service.
- Orchestrate retrieval and generation with a LangGraph decision layer that guards scope,

gates on retrieval confidence, reviews each draft, retries once and otherwise falls back safely.

- Study model compression and adaptation (fp16, 8-bit, NF4, LoRA, QLoRA, LoRA-then-NF4) of Qwen2.5-0.5B-Instruct, with and without retrieval, on quality, robustness, efficiency and perplexity.
- Support the farmer's real context: Hindi, Hinglish and Marathi input and output, photo-assisted symptom entry, five answer styles, source viewing, a workflow viewer and request monitoring.
- Deploy the system as a public application, and record honest limits so that results are not over-read.

1.4 Contributions

- **An end-to-end grounded advisory pipeline** with independent retrieval and generation services, a FastAPI endpoint and a Streamlit application deployed at `crop-disease-rag-advisor.streamlit.app`, using the hosted `openai/gpt-oss-20b` model through Groq as its generator.
- **A LangGraph decision layer** with fourteen nodes and six typed fallbacks whose review step rejects any dose number that does not appear verbatim in a cited chunk, and whose execution path can be inspected live in the application.
- **A verified evaluation dataset** (410 train, 150 test, 36 handwritten, 40 terminology, three long-prompt suites of 30 and 40 out-of-distribution questions, with a verifier that exits non-zero on leakage or inconsistency) and an evaluation harness that reports head-versus-rare F1, prompt-length robustness, refusal and dose-hallucination rates.
- **A controlled compression study** of six Qwen2.5-0.5B variants, reporting accuracy, grounding, citation correctness, memory, latency, throughput, energy, cost and perplexity, plus a reproducibility check across three independent training sessions.
- **A calibrated retrieval gate** (threshold 0.706, applied as 0.70) chosen on training and calibration questions only, with a full threshold sweep that exposes the safety-versus-coverage trade-off.
- **Product features driven by live review**: Hinglish/Hindi/Marathi handling, photo-to-symptom input, answer styles, unsupported-crop and need-crop fallbacks, a monitoring tab with CSV export and a regression suite of scripted live questions.

1.5 Technical Background

Retrieval-augmented generation: It conditions a generator on passages retrieved from an external corpus, which allows knowledge to be updated without retraining and answers to be attributed to sources [1]. In this project the corpus is a set of disease cards, and the generator is asked to output a JSON object whose `citations` field names the numbered passages it used.

Dense retrieval, BGE and FAISS: It maps queries and passages to vectors and ranks passages by similarity. The project uses the BGE-small-en-v1.5 embedding model (384 dimensions) [5], with the recommended query instruction prefix, and a FAISS flat index for exact nearest-neighbour search [6].

BM25 and Reciprocal Rank Fusion: They are used together. BM25 is a lexical ranking

function based on term frequency and document length [9]; it is strong when the query shares words with the passage, as farmers' symptom descriptions often do. Reciprocal Rank Fusion (RRF) merges ranked lists by summing $1/(k + \text{rank})$ over lists, which needs no score calibration between retrievers [7]; the project uses $k = 60$.

Qwen2.5: It is a family of open-weight language models [4]. The 0.5-billion-parameter instruction-tuned model is the base for all local variants because it fits a free-tier GPU in every precision studied.

LoRA, QLoRA and quantization: They are three related ways to adapt or shrink a model. Low-Rank Adaptation (LoRA) freezes the pretrained weights and trains small low-rank update matrices, here 8.8 million parameters at rank 16 [2]. QLoRA backpropagates through a frozen 4-bit NormalFloat (NF4) base with double quantization and paged optimisers, so the same adapter can be trained in far less memory [3]. LLM.int8() is the 8-bit matrix-multiplication scheme used for the int8 variant [10]. The study treats compression as a design axis to be measured, not assumed to be free.

LangGraph: It models an LLM application as a graph of nodes that read and write a shared state, with conditional edges for routing and cycles for retries [8]. It suits this project because the decision logic (guard, gate, review, retry, fallback) is a state machine that must be inspectable and testable.

1.6 Organisation of the Report

Section 2 describes the application workflow: the architecture, knowledge base, hybrid retrieval, prompting, the LangGraph decision layer, multilingual and photo support, the model variants and the user interface with its monitoring tab. Section 3 presents the experimental setup and the results, in the order dataset and retrieval, fine-tuning, generation quality and safety, efficiency and compression, observations from the live application, and a summary of findings. Section 4 concludes with the main conclusion, limitations and future work.

2. Application Workflow

2.1 System Architecture

The application is organised in four layers, shown in Figure 2.1. The **interface layer** accepts a question from a farmer through the Streamlit application, through the FastAPI endpoint `POST /diagnose`, or through a photograph that a vision model first converts into an editable symptom description. The **decision layer** is a LangGraph state machine that decides whether, how and with what evidence a question may be answered. The **service layer** contains two independent services: a stateless retrieval service and a generation service with swappable backends. The **data layer** holds the versioned knowledge-base artifacts, the model variants and the request log that feeds the monitoring tab.

Separating retrieval from generation is deliberate. Both services can run in-process (the default for the deployed application) or behind their own FastAPI endpoints (`retrieval.mode` and `generation.mode` in the configuration), so the cheap and fast retrieval step can be scaled independently of the expensive generation step. The generation service exposes one interface with several backends: Hugging Face Transformers for the local Qwen2.5-0.5B variants in fp16, int8 and NF4 (with optional LoRA or QLoRA adapters), an OpenAI-compatible client for hosted models and an Ollama client. The public application uses the OpenAI-compatible backend with

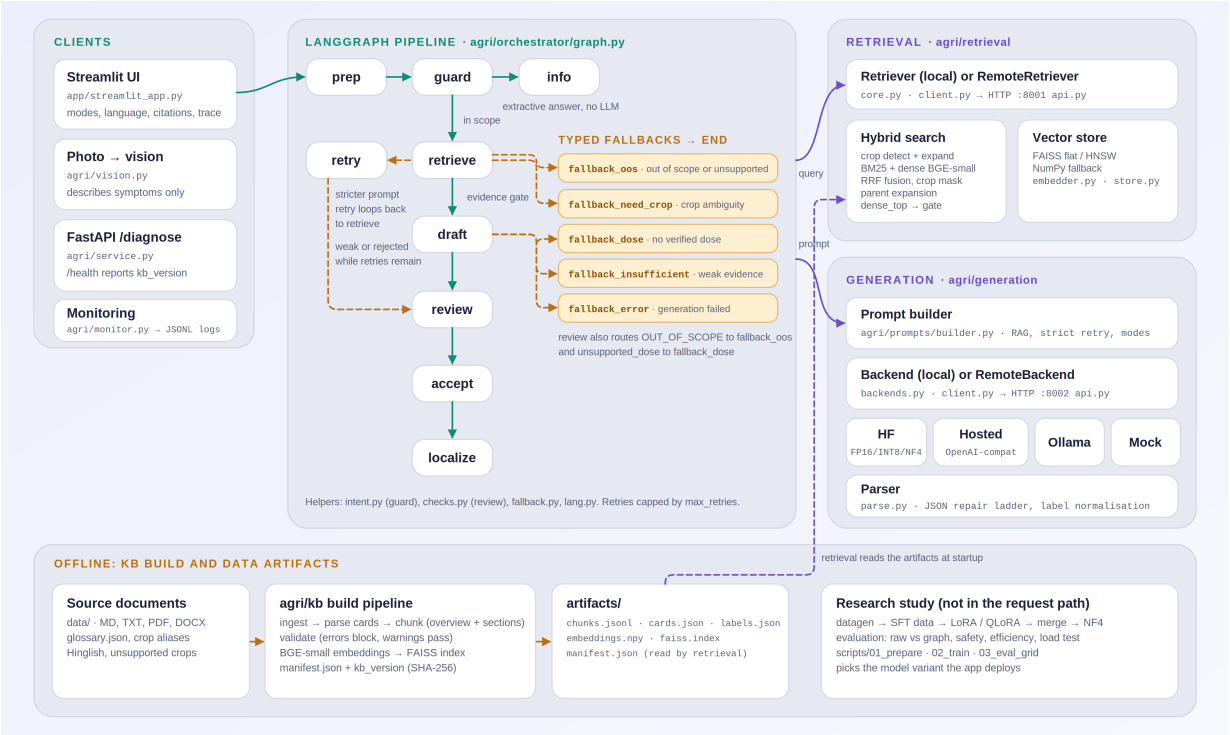


Figure 2.1: *Layered architecture of the grounded crop-disease advisor. Retrieval and generation are independent services coordinated by the LangGraph decision layer.*

`openai/gpt-oss-20b` [11] served by Groq, because the free hosting tier has no GPU; the local variants are used for the controlled study in Section 3.

A further design rule is that **one prompt builder serves inference, evaluation and supervised fine-tuning data construction**. Baseline (no retrieval) and RAG prompts share the same rules and output format, so any difference measured in Section 3 is attributable to retrieval and not to prompt wording.

2.2 Knowledge Base Construction

The knowledge base is built from seven Markdown documents: one per crop (rice, wheat, tomato, cotton, groundnut and sugarcane) and one general pesticide-safety document. Each disease is written as a card with fixed fields (type, aliases, pathogen, symptoms, favourable conditions, cultural, biological and chemical control). The build pipeline ingests and cleans the documents, parses the cards, splits them with structure-aware chunking, validates the result, embeds the chunks with BGE-small-en-v1.5 and writes a FAISS index. A manifest stores the SHA-256 hash of every source document and a content-derived version (`ad3be9cfa693`) that every response and log line carries, so any answer can be traced to the exact knowledge base that produced it.

Structure-aware chunking cuts a card at its section boundaries, so a chunk such as `tomato-early-blight::cultural_control::0` contains exactly one kind of information and records the start and end offsets of its text in the source file. These offsets let the interface highlight the cited sentence inside the full source document (Section 2.8). Section 3.2 compares this strategy with fixed-size "naive" chunking.

Table 2.1: *Knowledge-base and index statistics (build of 2026-10-30).*

Property	Value
Source documents	7 (6 crops + general pesticide guidance)
Cards / labels	19 cards; 18 disease or pest labels
Crops and labels	Rice 4 · Wheat 4 · Tomato 4 · Cotton 2 · Groundnut 2 · Sugarcane 2
Chunks	112 (structure-aware, $\leq$ 120 words, 20-word overlap, card header kept in chunk text)
Embedding model	BAAI/bge-small-en-v1.5, 384 dimensions, query instruction prefix
Vector index	FAISS flat (exact search); BM25 built over the same chunks
Versioning	`kb_version ad3be9cfa693`; SHA-256 per source document; validation report with 0 warnings
Provenance	Seed cards drafted from general agronomic knowledge; no numeric doses are stored

2.3 Hybrid Retrieval

The retrieval service (Figure 2.2) first understands the query. A glossary expands farmers' and regional names to canonical ones (for example "tikka" to Tikka leaf spot, "stripe rust" to yellow rust, "dithane" to mancozeb), detects the crop when exactly one crop is named, and, for queries longer than 150 words, scores overlapping 100-word windows with a stride of 50 so that filler text does not drown the diagnostic words. The expanded query is then searched twice: densely, by cosine similarity in the FAISS index, and lexically, by BM25. Chunks outside the detected crop are removed (general guidance chunks are kept) provided enough chunks remain.

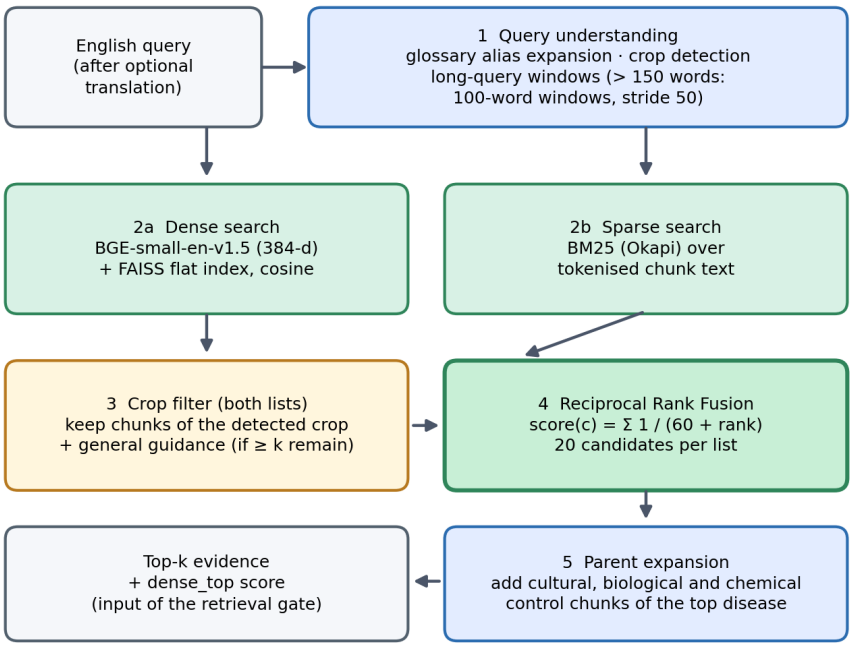


Figure 2.2: *Hybrid retrieval pipeline: query understanding, dense and sparse search, crop filtering, Reciprocal Rank Fusion and parent expansion.*

The two ranked lists, each with 20 candidates, are fused by Reciprocal Rank Fusion with $k = 60$, so a chunk ranked highly by either retriever rises without any need to calibrate BM25 scores against cosine similarities. **Parent expansion** then adds the management chunks (cultural, biological and chemical control) of the top disease, because a symptom description usually retrieves the symptom chunk while the farmer needs the treatment chunk. The service returns the top-k chunks together

with `dense_top`, the best dense similarity, which the decision layer uses as its confidence signal. The number of chunks passed to the generator depends on the answer style: three for the quick style and five for the detailed, prevention and organic styles.

2.4 Prompting and Generation

The generator receives a system prompt that lists the known crops and diseases, forbids invented pesticide names and doses, and requires **one JSON object** with the keys `crop`, `diagnosis`, `advice`, `citations` and `needs_expert`. In RAG mode the retrieved chunks are numbered [S1], [S2], ... and the model must cite the numbers it used, copy any dose exactly or give none, and answer `INSUFFICIENT_CONTEXT` when the passages do not identify the problem; a question that is not about these crops must be answered `OUT_OF_SCOPE`. A few-shot pair (one grounded rice example and one livestock refusal) fixes the format. Near-duplicate chunks are removed (Jaccard above 0.9) and a 600-word context budget is applied. Greedy decoding is used throughout (`do_sample=False` locally, temperature 0 for hosted models) so that results are repeatable.

Table 2.2: *Answer styles offered by the application and how each changes retrieval, generation and review.*

Style	Output	Chunks (k)	Max new tokens	Support factor*
Quick answer	3–5 short lines	3	220 (default)	1.0
Detailed guide	Labelled steps: confirm, do now, cultural, biological, chemical (label only), prevention	5	5096	0.8
Simple words	4–6 numbered steps, each under 15 words, no technical terms	3	800	0.9
Prevention plan	Seed, field hygiene, water and fertiliser, early signs to scout for	5	2048	0.8
Organic / low-chemical	Cultural and biological options only; no chemical sprays	5	2048	0.8

* The review step multiplies the minimum support score (0.45) by this factor, because longer styles paraphrase more.

2.5 LangGraph Decision Layer

The decision layer (Figure 2.3) is the main safety mechanism. It is implemented as a LangGraph `StateGraph` whose shared state carries the question, the detected language, the retrieval result, the draft, the review verdict, the attempt counter and a trace of every step. Conditional edges route each request to exactly one of three outcomes: an accepted, cited answer, a retry, or a typed fallback. The same graph is rendered live in the application so that a farmer or examiner can see which path a given question took.

The nodes behave as follows.

- **prep** detects Devanagari or Hinglish text and, for hosted backends, translates the question to English, because the embedding model is English-only. Translation is best-effort and falls back to the original text.
- **guard** is a cheap lexical first line of defence. Animal and human-health words win even when a crop word is present ("my rice spot has fever and is not eating"); a crop outside the covered six (about ninety names, including Hindi names such as *kela* and *aloo*) yields

`unsupported_crop`; a dose request is recognised **before** the crop requirement and, if no crop is named, never receives a number; a question that names no crop and too few symptoms yields `need_crop`; and "which diseases are favoured by ..." questions are routed to the extractive **info** node. The dose check is also run on the untranslated text, so translation cannot hide a dose request.

- **retrieve** calls the retrieval service. The routing function then applies the **gate**: if the best dense score is below 0.70 the request is retried once with an alias-expanded query and fewer chunks, and otherwise falls back with `insufficient`; a dose question whose retrieved chunks contain no dose falls back with `dose`; and a question with no crop whose two best diseases belong to different crops with near-equal scores (margin 0.03) asks for the crop.
- **draft** builds the prompt for the selected style, calls the generator and parses the JSON. A generation error routes to the `error` fallback, so an upstream outage never reaches the farmer as a stack trace.
- **review** decides whether the draft may be shown. It rejects invalid JSON, a label outside the known list, a crop that does not match the label, a missing or non-existent citation, a citation that does not name the diagnosed disease, advice with too little lexical support in the cited text, any dose or quantity not present in the retrieved context, and weak retrieval. A failed review is retried once with a strict prompt; a second failure becomes an `insufficient` or `dose` fallback, and a model-declared `OUT_OF_SCOPE` becomes an `out_of_scope` fallback.
- **accept** attaches the sources, the support score, a confidence level (high only when the dense score clears the gate by 0.08 and no near-tie exists) and an "also consider" list of diseases within the ambiguity margin. **localize** then rewrites the advice in Hindi, Hinglish or Marathi and discards the rewrite if it introduces any number that the English text did not contain.

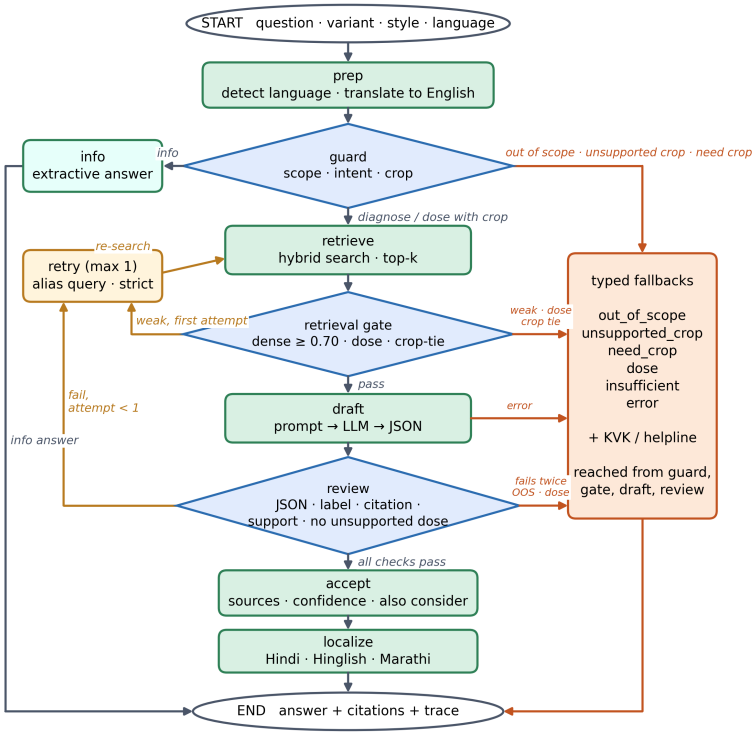


Figure 2.3: *LangGraph decision layer: nodes, conditional routes (diamonds), the single retry loop and typed fallbacks.*

2.6 Multilingual Input, Photo Input and Answer Control

Farmers often write Hindi in Roman letters with irregular spelling (for example *patton*, *rha*, *dhabbe*). Language detection uses the Devanagari script and a list of 86 Hinglish words; the translation prompt normalises such variants while preserving the crop, plant part, symptoms and numbers, and forbids diagnosing or adding advice. The answer language can be set to English, Hinglish, Hindi or Marathi, or follow the question; the answer is first produced and checked in English and only then rewritten, so retrieval, citations and review never depend on a translation of the answer. Translation and localisation run on the hosted backend; the small local models are used in English.

A farmer may instead upload a photograph. A vision model (served through the same OpenAI-compatible API) returns one or two sentences that describe only what is visibly wrong and is forbidden to name a disease or give advice. The description appears in the text box, where it can be edited, and then goes through the normal guard, retrieval and review path, so **the diagnosis is always grounded in the knowledge base and never in the image model**. Photos above 4 MB and images that do not show a plant are rejected with a clear message.

2.7 Model Variants, Fine-Tuning and Compression

All six local variants start from the same small model, Qwen2.5-0.5B-Instruct (about 0.5 billion parameters, roughly 1 GB in fp16) [4]. They differ only in how the model is shrunk or adapted:

- **fp16:** the original model in 16-bit numbers; the reference. About 1 GB on disk and about 1.2 GB of GPU memory in use.
- **int8:** weights squeezed to 8-bit numbers. About 0.6 GB on disk and about 0.8 GB in use, but slow on a T4 GPU.
- **NF4:** weights squeezed to 4-bit numbers. About 0.45 GB on disk (under half of fp16) and about 0.6 GB in use, with a small loss in quality.
- **LoRA:** fp16 model plus a small trained add-on (adapter). Only about 9 million weights (rank 16) are trained; the base stays frozen. Size about 1 GB.
- **QLoRA:** the same adapter, trained on top of the 4-bit NF4 model. About 0.5 GB in size, and training needs about half the memory of LoRA (about 5 GB against 10 GB).
- **LoRA→NF4:** the LoRA adapter merged into the fp16 model, then the merged model squeezed to 4 bits. About 0.45 GB.

Fine-tuning data come from the same prompt builder and the training split:

- **Size:** about 450 examples (405 for training, 45 for validation).
- **Mix:** about 260 grounded answers with citations, 40 “not enough context” cases, 115 answers without retrieval and 40 refusals for out-of-scope questions.
- **Training:** 3 epochs, learning rate 2×10^{-4}, effective batch of 16, on a free 15 GB T4 GPU (Hugging Face Transformers and PEFT).

2.8 User Interface, API and Monitoring

The Streamlit application has two tabs. The **Advisor** tab (Figure 2.4) offers an example-question dropdown, an optional photo upload, a text box, and sidebar controls for the model backend,

answer style and answer language. A result shows the likely problem, a confidence line ("strong match" or "possible match, please confirm", with the retrieval score against the 0.7 minimum and the answer-support score), any "also consider" diseases, the advice, an expert-consultation notice when the model or a near-tie asks for one, and clickable source cards. A source card shows the cited passage and a permalink; a toggle opens the entire source document with the cited span highlighted (Figure 2.5). A **Workflow** button opens the LangGraph execution viewer, in which the path taken by this particular request is drawn in green over the full graph, and a "How this answer was produced" trace lists each step. Thumbs-up and thumbs-down feedback with a reason is logged, and a per-session rate limit protects the hosted model.

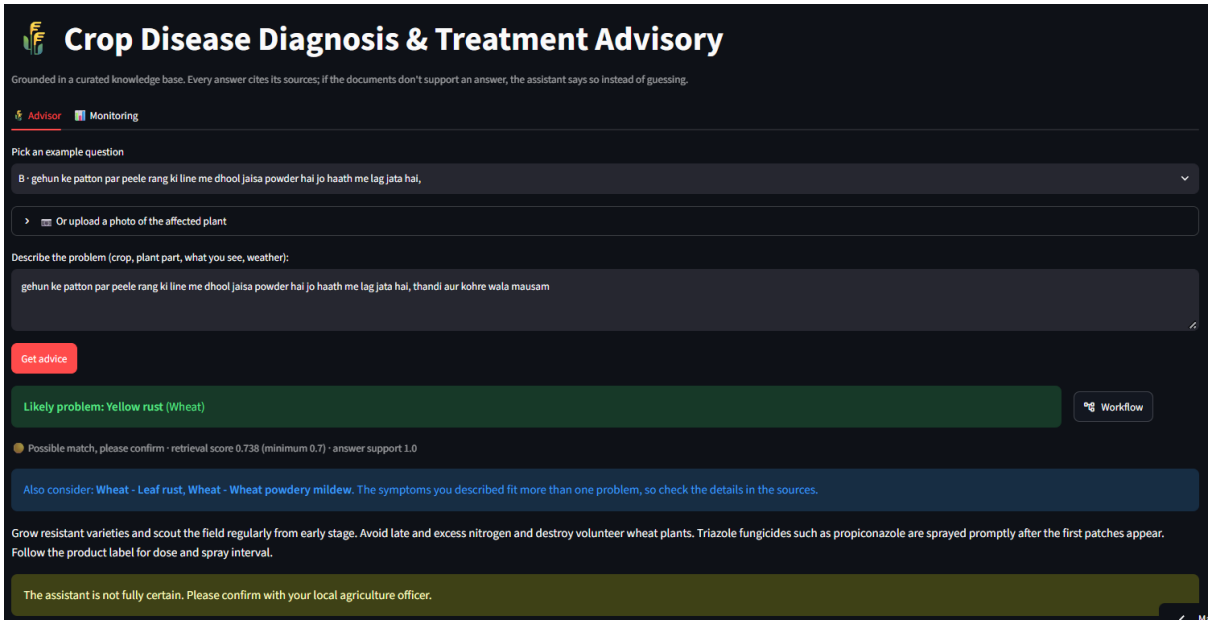


Figure 2.4: *Advisor tab answering a Hinglish question about wheat (yellow rust), with confidence line, "also consider" note, advice and expert notice.*

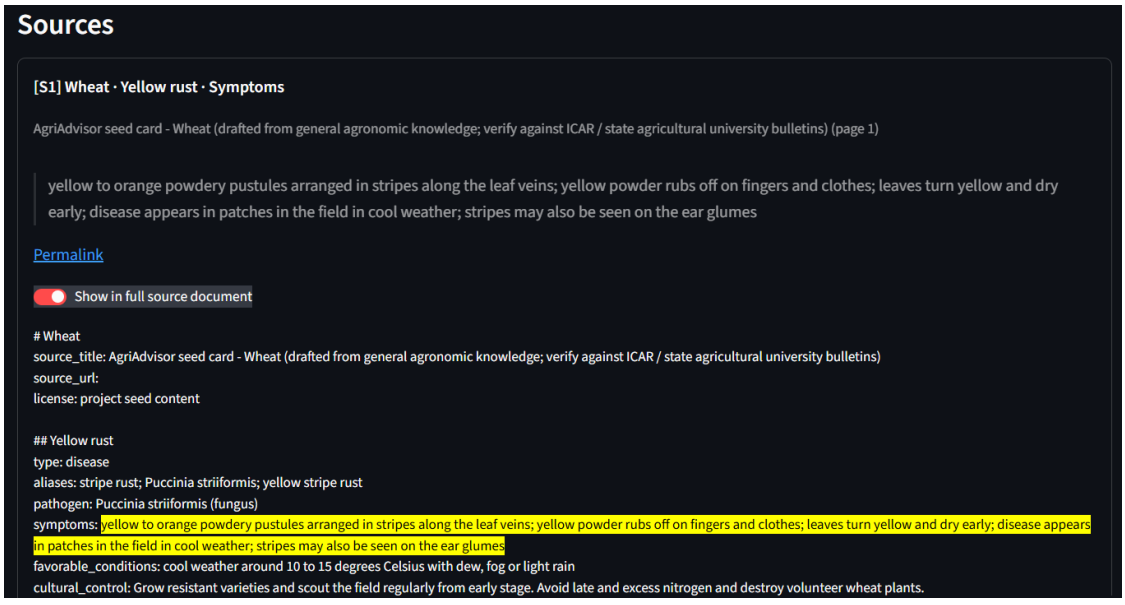


Figure 2.5: *Source viewer: the cited passage and the full source document with the supporting sentence highlighted.*

The **Monitoring** tab is described with its results in Section 3.8. The same pipeline is also exposed as a REST API: `GET /health` returns the knowledge-base version and `POST /diagnose` accepts

`text`, `variant` and `use_rag`, validates the length (5 to 4,000 characters), strips control characters, maps upstream failures to HTTP 502 and writes one log line per request.

3. Results and Observations

3.1 Experimental Setup

All quality, efficiency and perplexity numbers come from one complete, full-size evaluation run executed on a single Tesla T4 GPU (Kaggle, "Save & Run All", about 4.5 hours) with the retrieval gate fixed at 0.70 and greedy decoding. Every local variant was run with and without retrieval. The decision-layer ("graph") pipeline was run for QLoRA and for NF4, and results are reported for the **raw** model output (what the generator produced, scored directly) and for the **graph** output (after guard, gate, review, retry and fallback). A refusal or fallback counts as an incorrect diagnosis in accuracy, so accuracy is a deliberately strict measure for the graph pipeline. Retrieval evaluation, the gate sweep and the load test run on CPU. The hosted gpt-oss-20b generator of the live application is not part of this controlled grid (Section 4.2).

Table 3.1: *Evaluation suites. Every suite is generated or written independently of the training split and checked by the dataset verifier (0 errors).*

Suite	n	Purpose
train	410	Supervised fine-tuning source and gate calibration
test	150	In-distribution accuracy; 18 classes (8 head with 60 samples, 10 rare with 8 samples, 3 of them in test)
handwritten	36	Independently written, colloquial questions including Hindi transliterations; the least optimistic quality suite
terminology	40	Farmer and regional names, abbreviations and brand names for the same diseases and chemicals
long512 / long1024 / long2048	30 each	Symptom text embedded in diagnosis-neutral filler of about 512, 1024 and 2048 tokens
ood	40	Out-of-distribution: unseen crops 10, livestock 8, general 8, unsupported-dose requests 8, human health 6
ood_train	40	OOD questions used only to calibrate the gate

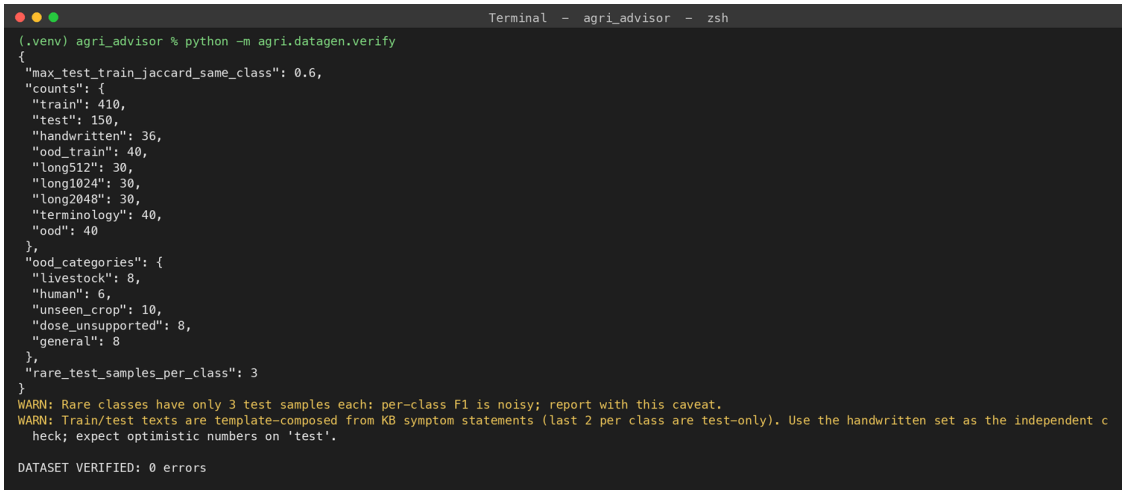


Figure 3.1: *Output of the dataset verifier: split sizes, OOD categories and the maximum same-class test/train Jaccard overlap of 0.6; "DATASET VERIFIED: 0 errors".*

The verifier also prints two warnings that shape how the results must be read. Rare classes have

only three test samples each, so per-class F1 is noisy; and train and test texts are composed from templates over the knowledge-base symptom statements, so **the test suite is expected to be optimistic** and the handwritten suite is the independent check.

Table 3.2: *Metrics used in Section 3.*

Metric	Definition
Accuracy, macro-F1	Exact diagnosis match over 18 classes (refusal = wrong); unweighted mean of per-class F1
Head / rare F1	Mean F1 over the 8 head and the 10 rare classes; exposes failures that the average hides
Refusal rate	Share of questions answered with a refusal label or fallback
Grounded	Lexical support: share of the answer's content words found in the cited passages
Citation correct	Cited passages exist and at least one belongs to the diagnosed disease
Dose violation	Share of in-scope answers containing a dose that does not occur in the retrieved context
OOD refusal / false answer	Share of OOD questions refused / answered with a diagnosis
Dose hallucination	Share of unsupported-dose questions on which a dose was invented
Recall@k, MRR, nDCG@5	Retrieval at disease level: a retrieved chunk is relevant if it belongs to the gold disease card
False accept / false fallback	Graph pipeline: wrong answer shown / in-scope question refused
TTFT, TPOT, throughput, energy, cost	Time to first token, time per output token, tokens per second (single and batch of 8), watt-hours per query, estimated USD per 1,000 queries at 0.35 USD per GPU hour

3.2 Dataset and Retrieval Results

Retrieval is evaluated first because it bounds everything downstream: if the right disease is not among the retrieved chunks, no generator can cite it. Table 3.3 compares dense, BM25 and hybrid retrieval on the four query styles that matter most, and Figure 3.2 summarises Recall@1 and MRR.

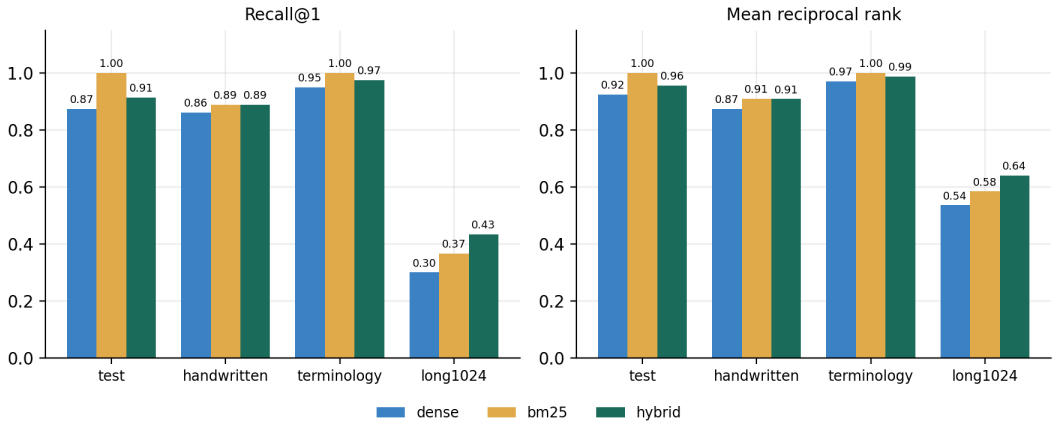


Figure 3.2: *Recall@1 (left) and mean reciprocal rank (right) of dense, BM25 and hybrid retrieval on four suites.*

Three observations follow. First, **the right disease is always retrieved within the top three on the test and terminology suites (Recall@3 = 1.000 for hybrid) and within the top five on 94% of handwritten questions**, so the retrieval step rarely starves the generator on realistic input; this is why the "retrieval hit rate" in later tables is 1.0 for test and terminology.

Table 3.3: *Retrieval quality by suite and configuration (disease-level relevance, top-5).*

Suite	Configuration	R@1	R@3	R@5	MRR	nDCG@5	ms/query
test (n=150)	dense (BGE+FAISS)	0.873	0.967	0.993	0.924	0.903	4.0
	BM25	1.000	1.000	1.000	1.000	0.956	0.2
	hybrid (RRF)	0.913	1.000	1.000	0.956	0.911	4.1
	hybrid, no crop filter	0.893	1.000	1.000	0.946	0.929	4.0
	hybrid, no alias expansion	0.913	1.000	1.000	0.956	0.911	4.0
handwritten (n=36)	dense (BGE+FAISS)	0.861	0.861	0.917	0.874	0.838	4.1
	BM25	0.889	0.917	0.944	0.910	0.888	0.2
	hybrid (RRF)	0.889	0.917	0.944	0.908	0.874	4.3
	hybrid, no crop filter	0.833	0.889	0.889	0.856	0.836	4.2
	hybrid, no alias expansion	0.889	0.917	0.944	0.908	0.874	4.3
terminology (n=40)	dense (BGE+FAISS)	0.950	1.000	1.000	0.971	0.950	4.6
	BM25	1.000	1.000	1.000	1.000	0.978	0.2
	hybrid (RRF)	0.975	1.000	1.000	0.988	0.966	4.8
	hybrid, no crop filter	0.975	0.975	0.975	0.975	0.944	4.7
	hybrid, no alias expansion	0.975	1.000	1.000	0.988	0.963	4.5
long1024 (n=30)	dense (BGE+FAISS)	0.300	0.733	0.933	0.536	0.636	162.3
	BM25	0.367	0.767	0.933	0.584	0.668	8.4
	hybrid (RRF)	0.433	0.833	0.933	0.640	0.697	167.9
	hybrid, no crop filter	0.633	0.700	0.833	0.698	0.706	171.5
	hybrid, no alias expansion	0.433	0.833	0.933	0.640	0.697	166.4

Second, BM25 is the strongest single retriever on the template-composed test and terminology suites (Recall@1 = 1.000), which is expected because those questions reuse the wording of the cards. It is not the strongest where wording differs: on handwritten questions dense and BM25 are close (0.861 vs 0.889 Recall@1), and on the 1024-token suite hybrid fusion is clearly best (Recall@1 0.433 against 0.300 dense and 0.367 BM25). Hybrid retrieval is therefore chosen for robustness, not because it wins every row. Third, accuracy of every method falls sharply with long inputs (Recall@1 on long1024 is 0.433 against 0.913 on test) and the latency rises from about 4 ms to about 165 ms because several windows are embedded.

The two query-understanding components were ablated. Removing the crop filter lowers Recall@1 on the test suite (0.913 to 0.893) and handwritten suite (0.889 to 0.833) and Recall@5 on handwritten and terminology questions, so it helps when the crop is named. On the long suite the opposite happens: without the filter Recall@1 is 0.633 against 0.433 with it, although Recall@5 is lower (0.833 against 0.933). Long prompts mention several crops or distracting words, so the crop detector can lock onto the wrong crop; this is a known weakness that is revisited in Section 4. Alias expansion produced identical metrics on these suites, which indicates that the hand-written aliases are rarely decisive for the retrieval of these particular questions, although they remain useful for brand names such as Dithane.

Structure-aware chunking gives the best Recall@1 (0.909) and MRR (0.946) whatever the size limit, because each card section is already below 60 words and so the limit never binds (112 chunks in all three rows). Naive fixed-size chunking creates far fewer, larger chunks (19 to 56) and, at 200 words, attains slightly higher Recall@5 and nDCG@5 simply because one large chunk carries much of a card. The structure-aware strategy was kept because it yields **section-level citations** (a farmer sees the exact symptom or control sentence that supports the advice) and the best top-1 ranking, which is what determines the single diagnosis shown.

Table 3.4: *Chunking strategy and chunk size ablation (BGE-small-en-v1.5, 186 questions).*

Chunking	Max words	Chunks	Recall@1	Recall@3	Recall@5	MRR	nDCG@5
structured	60	112	0.909	0.984	0.989	0.946	0.904
structured	120	112	0.909	0.984	0.989	0.946	0.904
structured	200	112	0.909	0.984	0.989	0.946	0.904
naive	120	28	0.882	0.995	1.000	0.939	0.928
naive	60	56	0.887	0.978	0.995	0.934	0.904
naive	200	19	0.860	0.995	1.000	0.928	0.947

3.3 Retrieval Gate Calibration and Service Load

The gate rejects a question when the best dense similarity is below a threshold. The threshold was calibrated on training and calibration-OOD questions only, giving 0.706 (in-scope accept rate 0.99, OOD reject rate 0.969; positive scores range from 0.671 to 0.941 and negative scores from 0.432 to 0.731) and was applied as 0.70. A full sweep over the evaluation suites (Figure 3.3, Table 3.5) shows what that choice costs.

Table 3.5: *Effect of the dense-score gate: share of in-scope questions accepted and share of OOD questions rejected.*

Gate	Accept test	Accept handwritten	Accept terminology	Accept long2048	Reject OOD (eval)	Reject OOD (calib.)
0.50	1.000	1.000	1.000	1.000	0.094	0.219
0.60	0.993	0.861	1.000	0.967	0.438	0.563
0.65	0.933	0.833	0.975	0.567	0.781	0.750
0.70	**0.820**	**0.833**	**0.700**	**0.400**	**0.969**	**0.875**
0.75	0.573	0.694	0.525	0.133	0.969	1.000
0.80	0.320	0.278	0.175	0.033	1.000	1.000

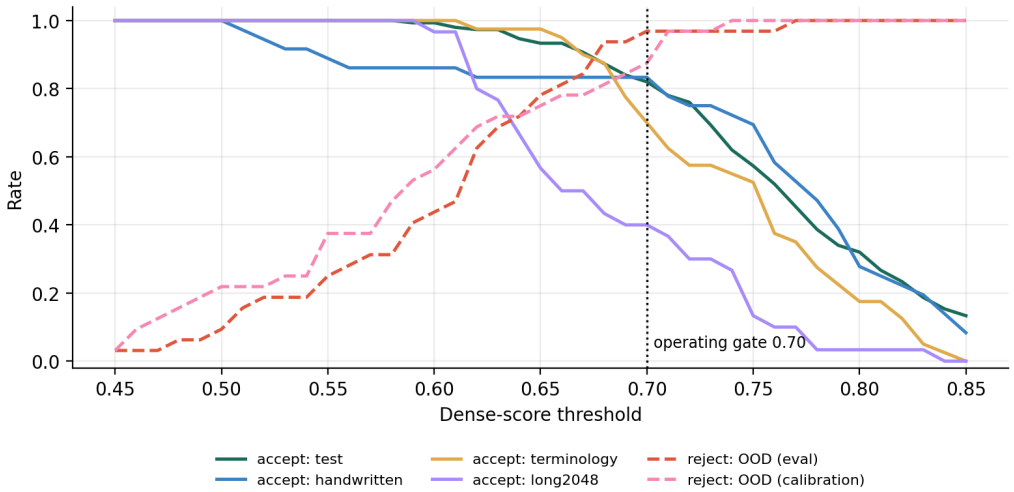


Figure 3.3: *Gate trade-off: acceptance of in-scope suites (solid) and rejection of OOD questions (dashed) as the dense-score threshold varies. The dotted line marks the operating point.*

At 0.70 the gate rejects 96.9% of the evaluation OOD questions but also withholds 18% of in-scope test questions, 17% of handwritten questions, 30% of terminology questions and 60% of the 2,048-token questions. This is the central safety-versus-coverage trade-off of the system. The operating

point was chosen to favour safety: at 0.65 and below OOD rejection is under 80%, while above 0.75 the gate becomes unusable for colloquial or long input. The service load test (Table 3.6, Figure 3.4) shows that retrieval itself is never the bottleneck: a single worker sustains 243 queries per second with a 5.4 ms p95 latency, and throughput saturates near 640 queries per second at eight concurrent clients, which is three orders of magnitude above what the generator can produce.

Table 3.6: *Retrieval service load test (local mode, CPU, 150 queries per level).*

Concurrency	Queries/s	p50 (ms)	p95 (ms)
1	242.8	3.9	5.4
4	477.2	7.9	13.3
8	644.3	11.8	18.1
16	634.8	20.7	42.1

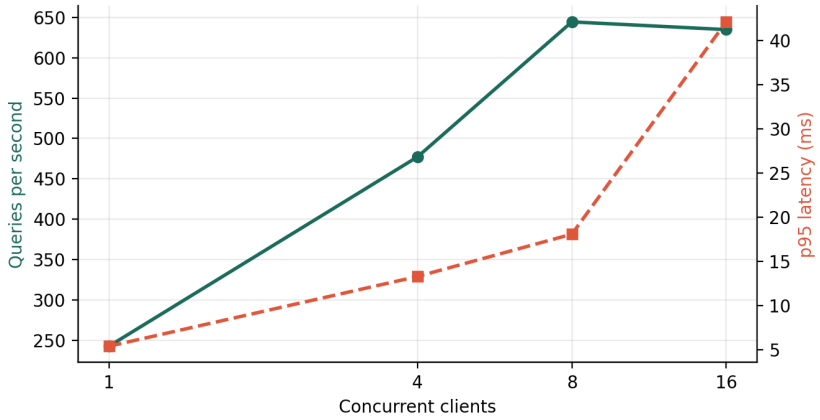


Figure 3.4: *Retrieval throughput and p95 latency against the number of concurrent clients.*

3.4 Fine-Tuning Results

Both adapters have the same 8,798,208 trainable parameters (1.75% of the LoRA model; 2.72% of the QLoRA model only because 4-bit packing shrinks the counted total). Table 3.7 reports the final training run used for the evaluation and, for comparison, two earlier sessions on a free Colab T4 with identical settings.

Table 3.7: *LoRA versus QLoRA training on a T4 GPU (3 epochs, rank 16). The **first row** group is the run behind all results in Section 3.*

Session	Mode	Train time (s)	Train loss	Val loss	Peak GPU (GB)
Kaggle (final)	LoRA (fp32 base)	951	0.196	0.178	10.17
	QLoRA (NF4 base)	631	0.397	0.355	5.43
Colab 1	LoRA (fp32 base)	528	0.122	0.136	8.98
	QLoRA (NF4 base)	856	0.130	0.134	3.30
Colab 2	LoRA (fp32 base)	553	0.122	0.133	8.98
	QLoRA (NF4 base)	903	0.130	0.133	3.30

QLoRA needs 47% less peak GPU memory than LoRA in the final run (5.43 against 10.17 GB) and 63% less in the Colab sessions (3.30 against 8.98 GB), which is the reason it can be trained on a free GPU. The time comparison is not consistent across sessions (QLoRA was slower on Colab

because of de-quantisation overhead and faster on Kaggle), and the final-run losses are higher than in the Colab sessions for both modes. Losses therefore differ between hardware sessions in the third decimal when the session is repeated and in the first decimal when the platform changes. As Section 3.5 shows, the higher QLoRA loss of the final run did not translate into lower accuracy, so training loss alone is not a reliable predictor of downstream quality in this study.

3.5 Generation Quality

3.5.1 Effect of retrieval

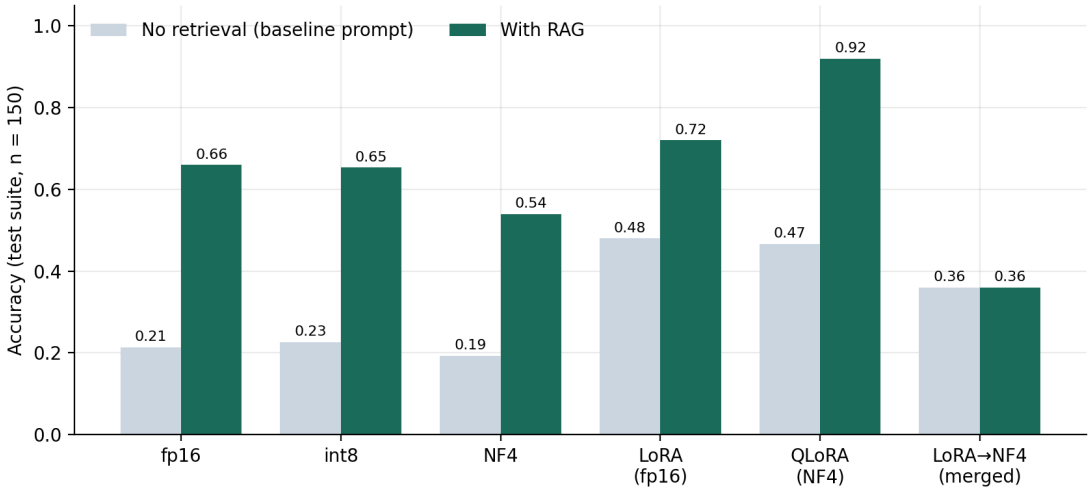


Figure 3.5: *Test accuracy of every local variant without retrieval and with RAG.*

Retrieval is the largest single improvement. Without it, the untuned models identify the disease correctly in only 0.21 (fp16), 0.23 (int8) and 0.19 (NF4) of test questions; with retrieval these rise to 0.66, 0.65 and 0.54, a gain of 35 to 45 points. Fine-tuning without retrieval reaches only 0.47 to 0.48, and its **rare-class F1 stays at 0.08 to 0.16**: the adapters teach the output format and the label set but cannot memorise ten rare diseases from eight examples each. The unusual case is the merged-and-quantised LoRA variant, which does not benefit from retrieval at all (0.36 with and without) because it refuses 61% of in-scope test questions once passages are present.

3.5.2 Quality by suite

Table 3.8: *Quality with retrieval on the three question suites (raw model output).*

Variant	Test acc.	Test macro-F1	Hand. acc.	Hand. macro-F1	Term. acc.	Parse OK (test)
fp16	0.66	0.60	0.61	0.57	0.78	0.99
int8	0.65	0.60	0.61	0.57	0.75	0.99
NF4	0.54	0.55	0.58	0.62	0.30	0.98
LoRA (fp16)	0.72	0.74	0.67	0.76	0.35	1.00
QLoRA (NF4)	0.92	0.92	0.83	0.83	0.72	1.00
LoRA→NF4 (merged)	0.36	0.41	0.22	0.28	0.07	1.00

QLoRA with retrieval is best on all three suites: 0.92 test accuracy (macro-F1 0.921), 0.83 on the independent handwritten questions and 0.72 on terminology questions. All of its test outputs

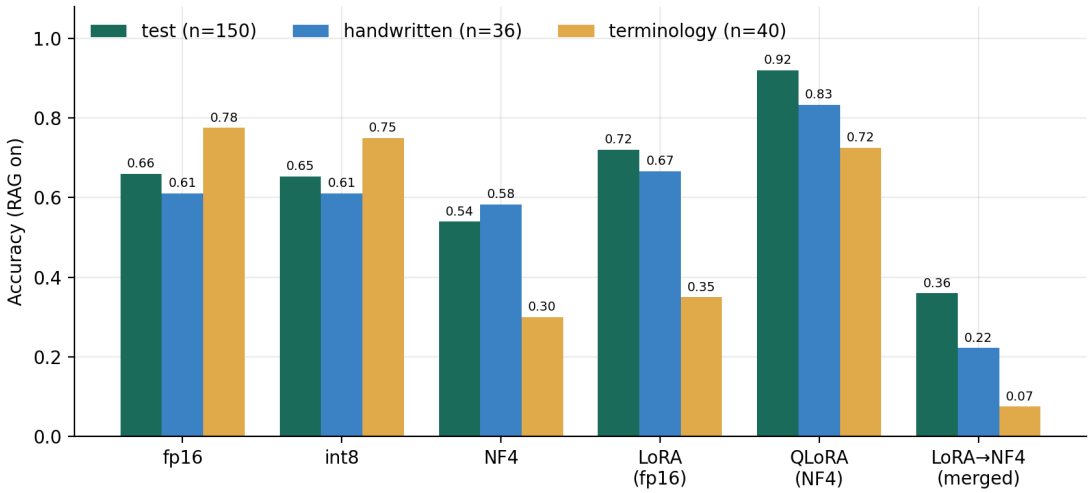


Figure 3.6: *Accuracy of each variant with retrieval on the test, handwritten and terminology suites.*

parse as valid JSON and none is a refusal, which means the 8% of errors are wrong diagnoses, not abstentions. The drop from test to handwritten (0.92 to 0.83) is the realistic expectation for unseen phrasing, and the further drop to terminology (0.725, with 25% refusals) shows that unfamiliar names remain the hardest input. The base models are less consistent: fp16 and int8 achieve their best score on the terminology suite (0.775 and 0.750), a pattern consistent with the lexical overlap between these questions and the cards, and NF4 is weakest there (0.300, with 60% refusals), which shows how readily the 4-bit base declines.

3.5.3 Grounding and citation correctness

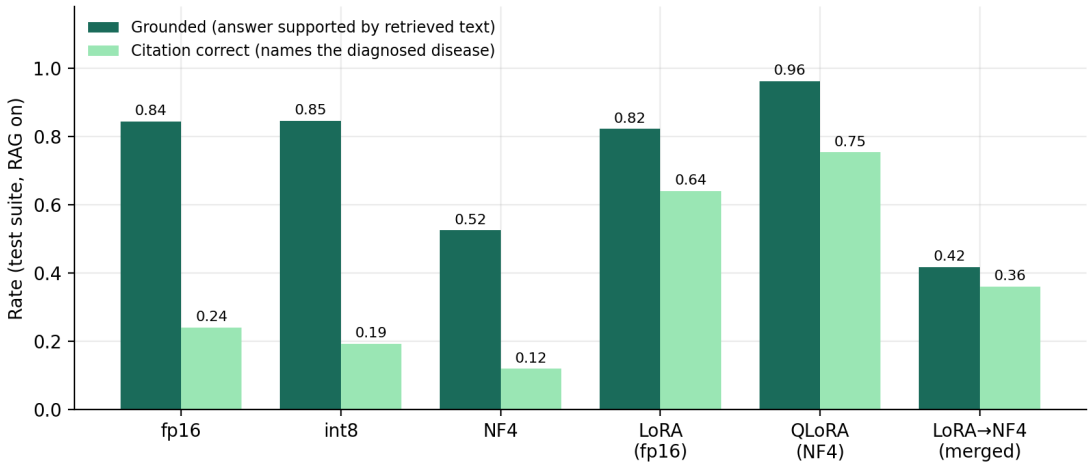


Figure 3.7: *Grounded-answer score and citation correctness on the test suite with retrieval.*

Grounding separates the base models from the adapted models more sharply than accuracy does. The base models produce advice that is lexically supported by the retrieved text (0.84 for fp16 and int8) but **cite the right disease only 24% (fp16) and 19% (int8) of the time**: they copy good text but attach the wrong passage number. LoRA raises citation correctness to 0.64 and QLoRA to 0.75, with a grounded score of 0.96. Fine-tuning therefore contributes mainly citation discipline and abstention behaviour, not disease knowledge. Citation correctness of 0.75 is still the weakest measure of the best variant, and it is the main reason the decision layer rechecks citations.

3.5.4 Robustness to prompt length

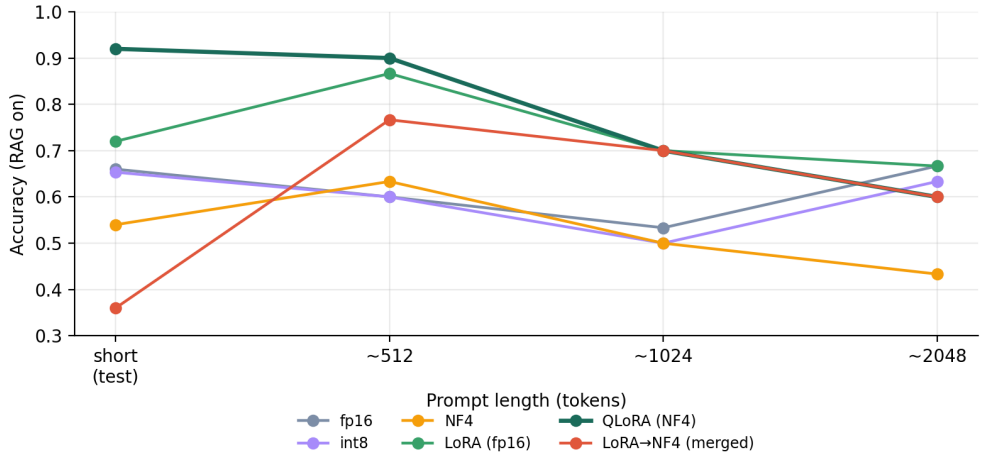


Figure 3.8: *Accuracy of each variant with retrieval as the prompt grows from a short question to about 2,048 tokens.*

QLoRA keeps 0.90 accuracy at 512 tokens, but falls to 0.70 at 1,024 and 0.60 at 2,048 tokens, which matches the retrieval finding that the diagnostic words are diluted by filler text. The same variant without retrieval falls from 0.47 to 0.13 at 2,048 tokens. The base fp16 and int8 models are flatter (0.53 to 0.67) but start lower. Long inputs are thus a real weakness for every variant, and the gate makes it visible by refusing 60% of 2,048-token questions rather than guessing.

3.6 Safety: Out-of-Distribution Questions and Doses

Table 3.9: *Out-of-distribution and dose behaviour with retrieval on (n = 40 OOD questions, of which 8 request an unsupported dose).*

Variant	Refusal rate	False-answer rate	Dose hallucination	Dose refusal
fp16	0.375	0.531	0.625	0.000
int8	0.400	0.500	0.500	0.000
NF4	0.775	0.063	0.375	0.125
LoRA (fp16)	1.000	0.000	0.000	1.000
QLoRA (NF4)	0.950	0.063	0.000	1.000
LoRA→NF4 (merged)	1.000	0.000	0.000	1.000
QLoRA + graph	0.975	0.031	0.000	1.000

This is where the study separates the variants most clearly. The retrieval-augmented base models **answer most out-of-distribution questions instead of refusing them**: fp16 refuses only 37.5% (false-answer rate 0.531) and int8 only 40%. They also invent a dose on 62.5% (fp16) and 50% (int8) of the unsupported-dose questions. Fine-tuning corrects this: LoRA and QLoRA refuse 100% and 95% respectively and never invent a dose, and the graph pipeline lifts QLoRA to 97.5% (one false answer among 40). The NF4 base is conservative (77.5% refusal) but still hallucinates a dose on 37.5% of dose questions, and its refusals reflect general over-caution rather than discrimination, as its 24% refusal of in-scope test questions shows. The merged variant refuses 100% of OOD questions only because it refuses nearly everything.

Without retrieval, NF4 states an unsupported dose in 35% of its in-scope answers and fp16 and int8

in 7%, which is exactly the hazard that motivates the project. With retrieval no variant violates the dose rule on the test suite (int8 shows 0.7% in total), because the prompt forbids numbers outside the passages and the cards themselves contain no numeric doses. The decision layer enforces the same rule mechanically, so a violation that does occur is rejected rather than displayed.

3.7 Efficiency and Compression Trade-offs

Table 3.10: *Efficiency of the six variants on a Tesla T4 (20 prompts, about 931 input and 80–100 output tokens each, RAG on).*

Metric	fp16	int8	NF4	LoRA (fp16)	QLoRA	LoRA→NF4
Disk (MB)	988	632	457	988	539	457
GPU memory after load (GB)	0.988	0.631	0.457	0.988	0.493	0.457
Peak GPU, single query (GB)	1.156	0.798	0.625	1.156	0.660	0.624
Peak GPU, batch 8 (GB)	1.384	1.888	1.710	1.384	1.746	0.856
TTFT p50 (ms)	45	334	102	45	183	82
TPOT (ms/token)	33.0	160.8	53.2	34.3	93.2	47.9
Latency per query (s)	2.63	12.82	4.30	3.25	9.58	3.90
Throughput, single (tok/s)	30.2	6.1	18.6	29.1	10.6	20.7
Throughput, batch 8 (tok/s)	167	28	86	168	59	110
Energy per query (Wh)	0.040	0.122	0.048	0.048	0.100	0.044
Cost per 1,000 queries (USD)	0.256	1.246	0.418	0.316	0.932	0.379
Held-out perplexity	33.4	33.2	47.7	32.8	46.7	49.3

Compression buys memory, not speed, on this GPU. NF4 shrinks the model from 988 MB to 457 MB on disk (a 54% reduction) and lowers peak single-query GPU memory from 1.16 to 0.63 GB, but a query takes 4.3 s instead of 2.6 s and throughput at batch 8 halves (86 against 167 tokens per second), because every matrix multiplication must dequantise 4-bit weights. At batch 8 the NF4 model even peaks higher than fp16 (1.71 against 1.38 GB). int8 is the worst of both worlds on a T4: 12.8 s per query, 6 tokens per second and 1.25 USD per 1,000 queries, five times the cost of fp16, with little memory benefit. The best efficiency is fp16 (0.26 USD per 1,000 queries, 0.040 Wh per query). QLoRA is slower (9.6 s per query, 0.93 USD) because the adapter is evaluated beside a 4-bit base, and its 539 MB footprint includes an adapter of roughly 82 MB.

Perplexity shows the quality price of 4-bit quantisation: the held-out value rises from 33.4 (fp16) to 47.7 (NF4), whereas int8 (33.2) is indistinguishable from fp16 and LoRA (32.8) is marginally better. QLoRA (46.7) recovers only a little of the NF4 loss in perplexity, yet it gives the largest task gains, which confirms that perplexity on general agricultural text does not predict task accuracy. The perplexity set is small (480 tokens), so differences of one or two points should not be interpreted.

Figure 3.11 combines quality and cost. QLoRA reaches rare-class F1 of 0.917 within 0.66 GB of peak memory, whereas LoRA in fp16 needs 1.16 GB for 0.684 and fp16 needs the same memory for 0.526. If memory is the binding constraint, QLoRA is the only configuration in the study that does not give up rare-class quality; if latency or cost per query is the constraint, fp16 with retrieval and the decision layer is the efficient alternative. Our original decision rule (ship the base model with RAG if it meets the thresholds) is therefore **not** met by the base variants on safety grounds, because they answer about half of the OOD questions.

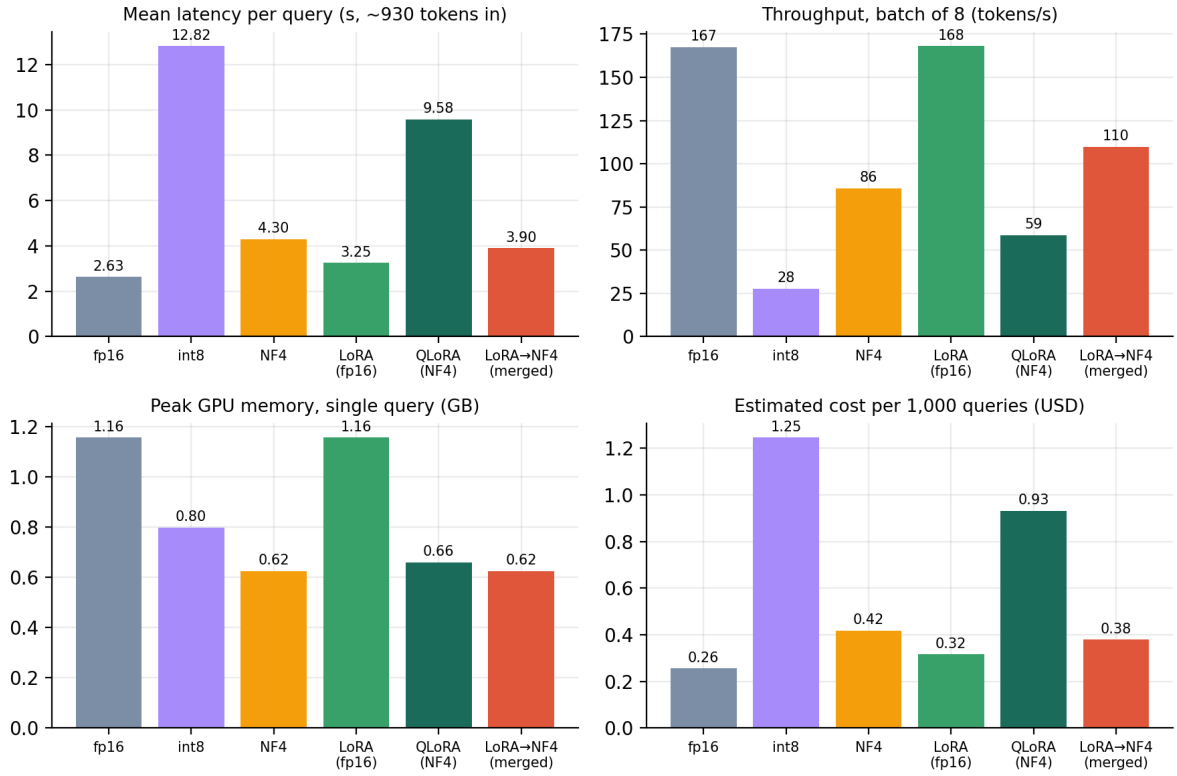


Figure 3.9: *Latency, batch throughput, peak memory and estimated cost of the six variants.*

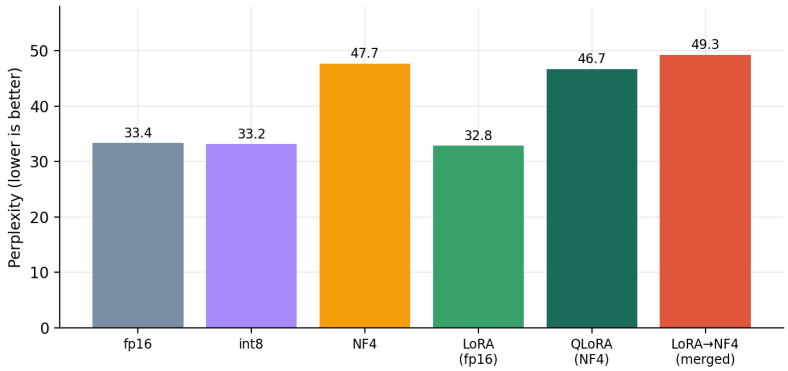


Figure 3.10: *Perplexity on 25 held-out agricultural texts (480 tokens); lower is better.*

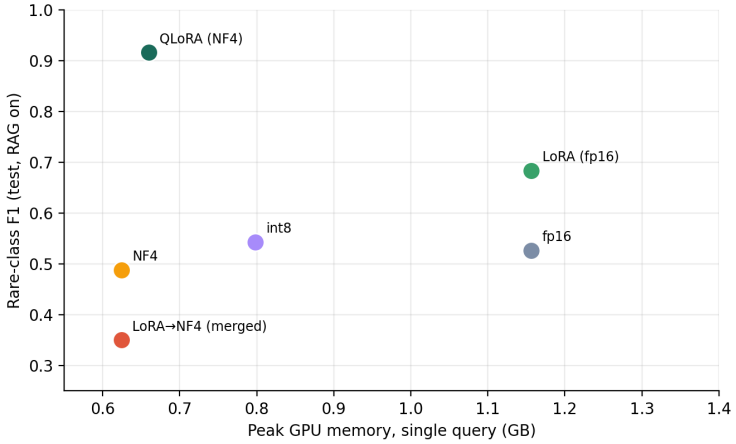


Figure 3.11: *Rare-class F1 against peak GPU memory: QLoRA is the only variant that is both small and strong on rare classes.*

3.8 Observations from the Live Application

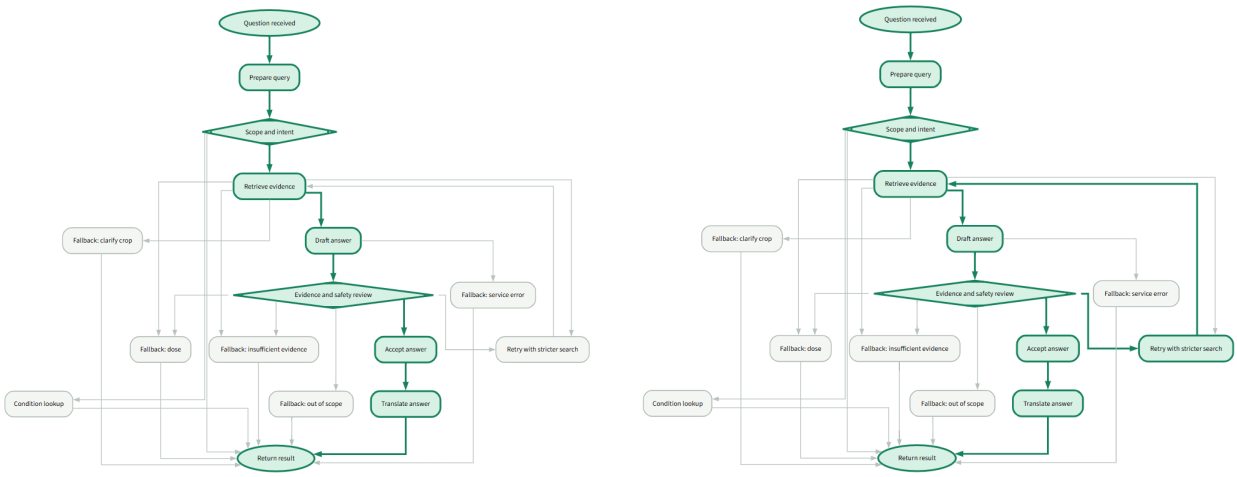
The deployed application (https://crop-disease-rag-advisor.streamlit.app) was exercised with a set of review questions that covers accepted answers, a retry, a photo, and each kind of fallback. The screenshots below were taken from this live instance, whose generator is `openai/gpt-oss-20b` on Groq. They are qualitative evidence that the decision layer behaves as designed on the hosted model; the quantitative evidence for the local models is in the preceding sections.

3.8.1 Accepted answers and the retry path

A Hinglish question about wheat (shown in the Advisor figure of Section 2) is translated, retrieved, drafted, reviewed and accepted without a retry, and the answer is returned with a possible-match notice because the retrieval score of 0.738 is only 0.038 above the gate and leaf rust and powdery mildew are within the ambiguity margin. A question about rice seedlings with spindle spots under humid, cool nights is answered as rice blast with a **strong match** (retrieval score 0.781). Its workflow (right panel below) shows the retry branch: the first draft failed the review, the request was re-searched with an alias-expanded query and a strict prompt, and the second draft was accepted. The execution viewer makes this visible without reading logs.



Figure 3.12: *Rice-blast question answered as a strong match (retrieval score 0.781, answer support 1.0).*



(a) *Accept path for the Hinglish yellow-rust question* **(b)** *Retry path for the rice-blast question*

Figure 3.13: *LangGraph execution viewer for two accepted requests. Green nodes and arrows show the path taken; grey nodes are the other possible routes.*

3.8.2 Photo-assisted entry

In the photo path (next figure) a tomato-leaf image is converted into a symptom description (distinct yellowing between the veins, curled and distorted leaflets, no visible insects). The description is

placed in the editable text box and passed through the normal pipeline, which returns tomato leaf curl virus with retrieval score 0.76, lists early blight and bacterial wilt as "also consider" and advises insect-proof nets, uprooting infected plants and yellow sticky traps. The retrieval score of 0.76 is above the gate by only 0.06, so the confidence line asks the farmer to confirm; the image model never names the disease.

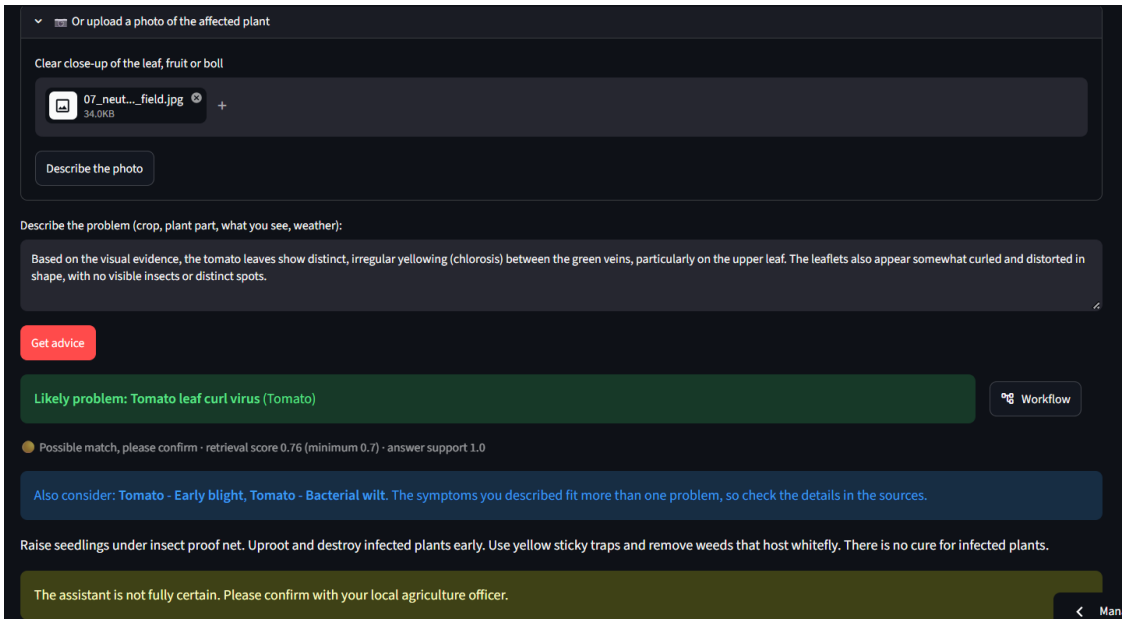


Figure 3.14: *Photo upload converted into an editable symptom description, followed by the grounded diagnosis.*

3.8.3 Safe fallbacks

Two fallbacks are shown. A request for the **dose** of propiconazole per hectare for wheat yellow rust is refused: the retrieved chunks contain no verified dose, so the dose fallback explains that doses depend on the product, crop stage and spray volume and refers the farmer to the product label and the Kisan Call Centre. A question about **banana** leaves is stopped by the guard before any retrieval, because banana is not among the six covered crops, and the message lists the covered crops. The execution paths confirm the different depths of the two decisions: the dose request reaches retrieval and then branches to the dose fallback, whereas the unsupported crop leaves the graph directly after the scope guard, at almost no cost.

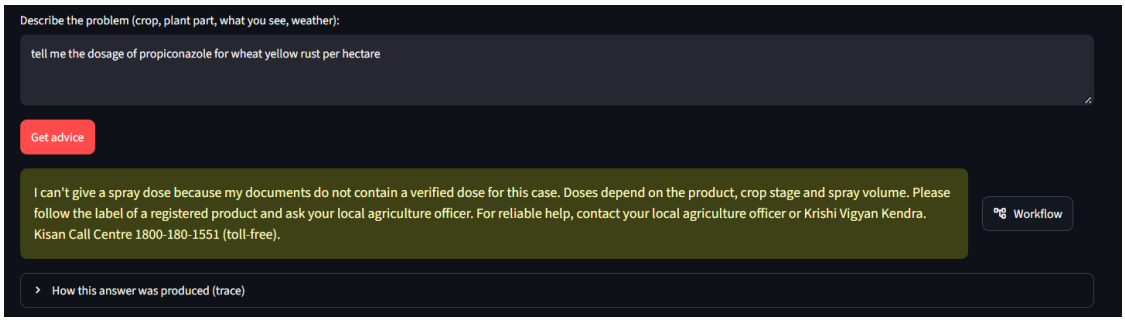


Figure 3.15: *Dose request refused with a verified-dose explanation and helpline.*

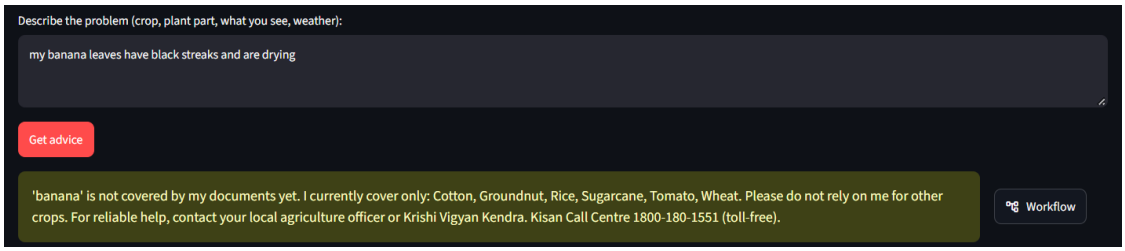
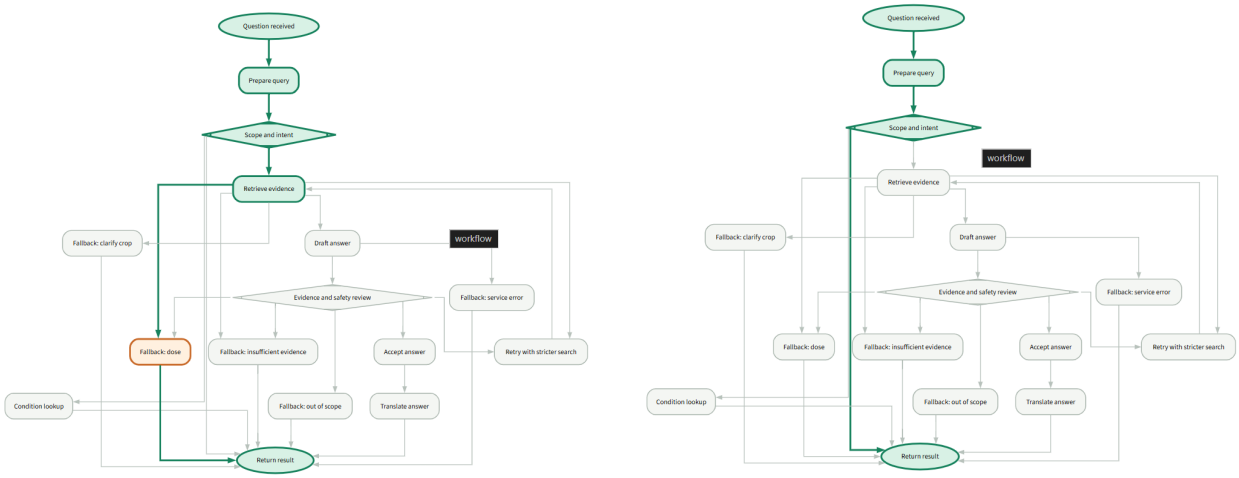


Figure 3.16: *Unsupported crop (banana) stopped by the scope guard.*



(a) *Dose fallback: retrieval, then fallback* **(b)** *Unsupported crop: guard, then return*

Figure 3.17: *Execution paths of the two fallbacks, as drawn by the workflow viewer.*

3.8.4 Monitoring tab

The Monitoring tab turns the request log into an operations view. Its health panel reports the knowledge-base version (`ad3be9cfa693`), the number of chunks (112), the retrieval gate (0.7), the number of diseases covered (18) and whether a language-model key is configured, and a button runs a live health check (in the capture: retrieval OK in 9 ms with a top dense score of 0.77, language model OK in 232 ms). The request panel shows the number of requests, answered and fallback counts, median and p95 latency and the retry rate, together with a bar chart of outcomes by type and a line chart of latency per request. A filterable table lists every logged request with its question, status, fallback type, diagnosis, response, number of sources, style, language, variant, intent, attempts, latency and review reasons, and can be downloaded as CSV.

In the captured session the application had logged 13 requests: 7 answered (54%) and 6 fallbacks (one dose, two insufficient evidence, two need-crop and one unsupported crop), with a median latency of 825 ms, a p95 latency of 1,610 ms and a retry rate of 31%. The low answered share must not be read as a quality figure, because the session was a deliberate probe of fallback behaviour. The numbers do show the intended operating profile of the hosted pipeline: answers in well under two seconds for most requests, fallbacks returned in a few milliseconds because they bypass generation, and every non-answer carrying a machine-readable reason that can be reviewed.

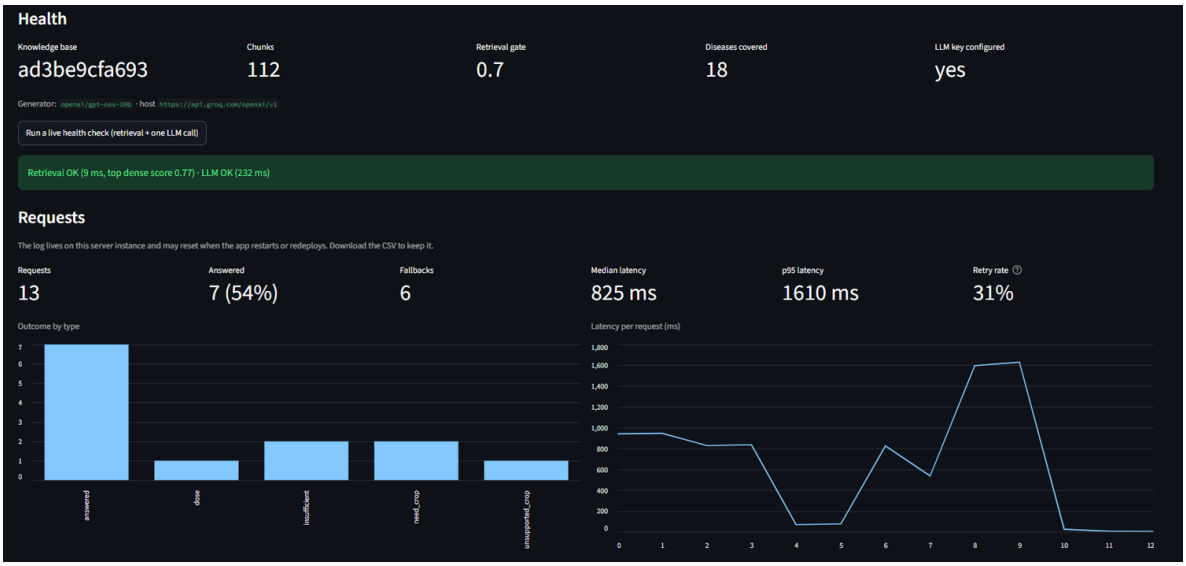


Figure 3.18: *Monitoring tab: knowledge-base health, live health check, request statistics and charts.*

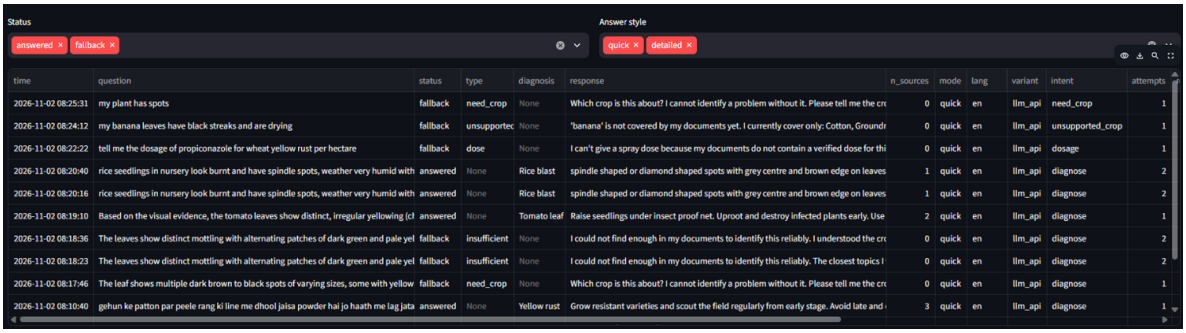


Figure 3.19: *Monitoring tab: filterable request log with question, status, type, diagnosis, response and metadata columns.*

3.9 Summary of Findings

- **Retrieval helps:** it is the largest single gain. Untuned models rise from 0.19–0.23 to 0.54–0.66 test accuracy and stop inventing doses (Table 3.8).
- **Best local variant:** QLoRA with retrieval reaches 0.92 test and 0.83 handwritten accuracy, 0.917 rare-class F1 and 0.75 citation correctness within 0.66 GB of peak memory (Table 3.8, Figure 3.11).
- **Safety on out-of-distribution questions and doses:** base variants answer 50–53% of OOD questions and invent doses on 50–63% of dose requests; QLoRA refuses 95% (97.5% with the graph) and never invents a dose (Table 3.9).
- **Cost of the decision layer:** about 16 accuracy points on the test suite (20% of in-scope questions withheld) in exchange for a 4% false-accept rate and no wrong answer on handwritten questions.
- **Quantisation is not free:** NF4 halves disk size and peak memory but raises perplexity from 33 to 48 and is 1.6× slower on a T4; int8 is the slowest and most expensive (Table 3.10).
- **Hybrid retrieval pays off:** it is never the worst and is best on long prompts, while BM25 wins on templated text. Structure-aware chunking gives the best top-1 accuracy and section citations (Tables 3.3 and 3.4).

- **Reproducibility:** base variants are deterministic; QLoRA stays within 1.3 accuracy points over three trainings, whereas LoRA→NF4 is unstable.

4. Conclusion

4.1 Conclusion

This project built a crop-disease advisor whose answers a farmer can check before acting on them. It combines hybrid dense and lexical retrieval over a versioned, section-chunked knowledge base with a LangGraph decision layer that guards scope, gates on retrieval confidence, reviews citations, support and doses, retries once and otherwise declines with a typed, helpful fallback, and it is deployed as a Streamlit application and a REST API. The study of six Qwen2.5-0.5B-Instruct variants shows that retrieval is what makes a small model usable (35 to 45 points more accuracy), that fine-tuning adds citation discipline and refusal behaviour more than knowledge, that the decision layer turns a good model into a safe system at the price of withholding about a fifth of in-scope questions, and that compression is a trade-off, not a free gain. QLoRA with retrieval was the best local variant: 0.92 test accuracy and 0.917 rare-class F1 within 0.66 GB of GPU memory.

4.2 Limitations

The live application uses the hosted `openai/gpt-oss-20b` model, which was not benchmarked in the controlled grid, so the result that QLoRA is the best local variant must not be extended to it. Accuracy and retrieval degrade for inputs of 1,000 tokens or more, and the 0.70 gate withholds about 60% of 2,048-token questions and 30% of terminology questions. Retrieval uses an English-only embedding model, so Hindi, Hinglish and Marathi depend on LLM translation that native speakers have not validated, and the scope guard and review step rely on hand-written word lists and lexical support scores that cannot detect the right words used with the wrong meaning.

4.3 Future Work

- **Replace the seed knowledge base** with reviewed ICAR, state university and KVK bulletins, and extend coverage to more crops and regions.
- **Evaluate the hosted generator** in the same grid and compare it with the best local model on quality, latency and cost.
- **Use a multilingual embedding model** and an optional re-ranker to remove the dependence on translation and improve long-query retrieval.
- **Collect real farmer queries** with KVK partners to build a larger independent test set and run an expert and farmer evaluation.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474. arXiv:2005.11401.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[3] T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. arXiv:2305.14314.

[4] A. Yang et al. (Qwen Team), "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[5] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings (BGE)," in *Proc. ACM SIGIR*, 2024. arXiv:2309.07597.

[6] J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. arXiv:1702.08734.

[7] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. ACM SIGIR*, 2009, pp. 758–759.

[8] LangChain Inc., "LangGraph: Build resilient language agents as graphs," software documentation, `https://github.com/langchain-ai/langgraph` (accessed Nov. 2026).

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[10] T. Dettmers, M. Lewis, Y. Belkada and L. Zettlemoyer, "LLM.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022. arXiv:2208.07339.

[11] OpenAI, "gpt-oss-120b and gpt-oss-20b model card," arXiv:2508.10925, 2025.

[12] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.